CS110 (Principles of CS)
RECITATION 7
Python – 3

RECITATION INSTRUCTOR – Dharm Patel
SECTION – 3 & 4

Agenda

1. **IPO-S, Flowcharts & Pseudocode**
   How a plan turns into a program

2. **User Input & Type Conversion**
   input() gives a string – int() / float()

3. **Relational & Boolean Operators**
   <, <=, ==, != … and, or

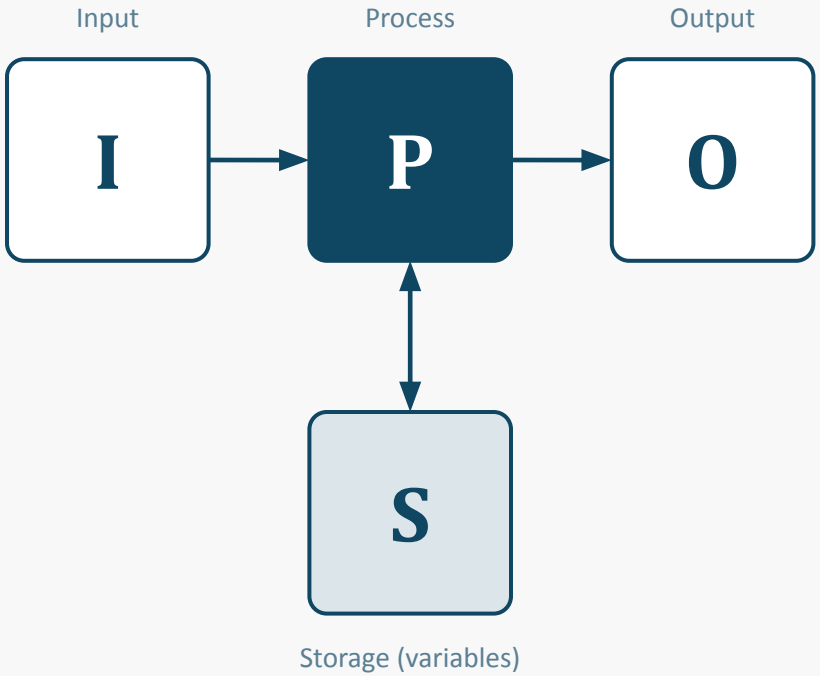
4. **IF, IF-ELSE & Nested IF-ELSE**
   Letter grade program, two ways

5. **Math in Python**
   import math, sqrt, ceil, floor

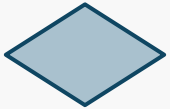
6. **Ending a Program Early**
   quit() + Python_03 assignment walkthrough

IPO-S Model

- **IPO-S** is the basic design of how any program works
- **Input –** data the program needs from the user
- **Process –** the calculations and decisions done on that data
- **Output –** the result shown to the user
- **Storage –** where values are kept for later use → our variables
- Always plan the IPO-S first, then write the code



Flowchart Symbols

	**Start / End**	Oval – where the program begins or stops
	**Input / Output**	Parallelogram – get input or display output
	**Process**	Rectangle – a calculation or assignment
	**Decision**	Diamond – a yes/no question (an if condition)
	**Flow**	Arrow – shows the order of the steps

From Plan to Program

IPO-S

What goes in, what we do, what comes out, which variables

→

Pseudocode

The steps written in plain English, in order

→

Flowchart

The same steps drawn as a picture

→

Python

Each step becomes one or more lines of code

Example: BMI calculator (pseudocode)

```
Step 1: Get the input for height
Step 2: Get the input for weight
Step 3: Calculate BMI = weight / height²
Step 4: Display BMI
```

Tracing tip: pick sample inputs and walk through every step of the pseudocode / flowchart by hand. If the plan gives the right answer, the code will too.

User Input & Type Conversion

- **`input()`** always returns a **string**
- Convert it before doing math:
  - **`int()`** → whole numbers (age, grade)
  - **`float()`** → decimals (price, length)
- Forgetting to convert = string joining, not adding!

```
priceStr = input("Price: ")
price = float(priceStr)
```

Without conversion

```
x = input("Enter x: ")     # "5"
y = input("Enter y: ")     # "3"
print(x + y)               # 53
```

With conversion

```
x = int(input("Enter x: "))
y = int(input("Enter y: "))
print(x + y)               # 8
```

Relational Operators

Operator	Meaning	Example (x = 7)	Result
<	less than	`x < 10`	`True`
<=	less than or equal	`x <= 7`	`True`
>	greater than	`x > 7`	`False`
>=	greater than or equal	`x >= 5`	`True`
==	equal to	`x == 7`	`True`
!=	not equal to	`x != 7`	`False`

Remember

- Every comparison gives back True or False
- = stores a value
- == compares two values
- **Mixing them up is the #1 bug in if statements!**

Boolean Operators: and, or

and – BOTH must be True

```
grade >= 80 and grade < 90

# 85 → True and True  → True
# 95 → True and False → False
```

or – AT LEAST ONE is True

```
day == "Sat" or day == "Sun"

# "Sun" → False or True → True
# "Mon" → False or False → False
```

A	B	A and B	A or B
T	T	**True**	**True**
T	F	**False**	**True**
F	T	**False**	**True**
F	F	**False**	**False**

We will use and in the letter-grade program to check the B range: 80 ≤ grade < 90

IF and IF-ELSE Statements

if

```
if condition:
    # runs only when
    # condition is True
    statement
```

Use when you only need to do something in one case

if – else

```
if condition:
    statement_A   # True
else:
    statement_B   # False
```

Exactly one of the two blocks runs – never both, never neither

Watch out: the colon (:) at the end, and the 4-space indentation – Python uses indentation to know what is inside the if.

Problem: Letter Grade

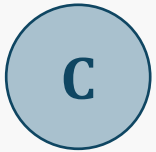
A person will input a numerical grade. The equivalent letter grade will be output.



90 – 100



80 – 89



79 and below

IPO-S

Input: numerical grade

Process:

```
if grade >= 90 → letterGrade = A
if grade >= 80 and grade < 90 → B
if grade < 80 → letterGrade = C
```

Output: the letter grade

Storage: variables grade, letterGrade

Program 1: Separate IF Statements

```
# Recitation problem 1
grade = int(input("Enter the integer numeric grade: "))

if grade >= 90:
    letterGrade = "A"
if grade >= 80 and grade < 90:
    letterGrade = "B"
if grade < 80:
    letterGrade = "C"

print("Your letter grade is ", letterGrade)
```

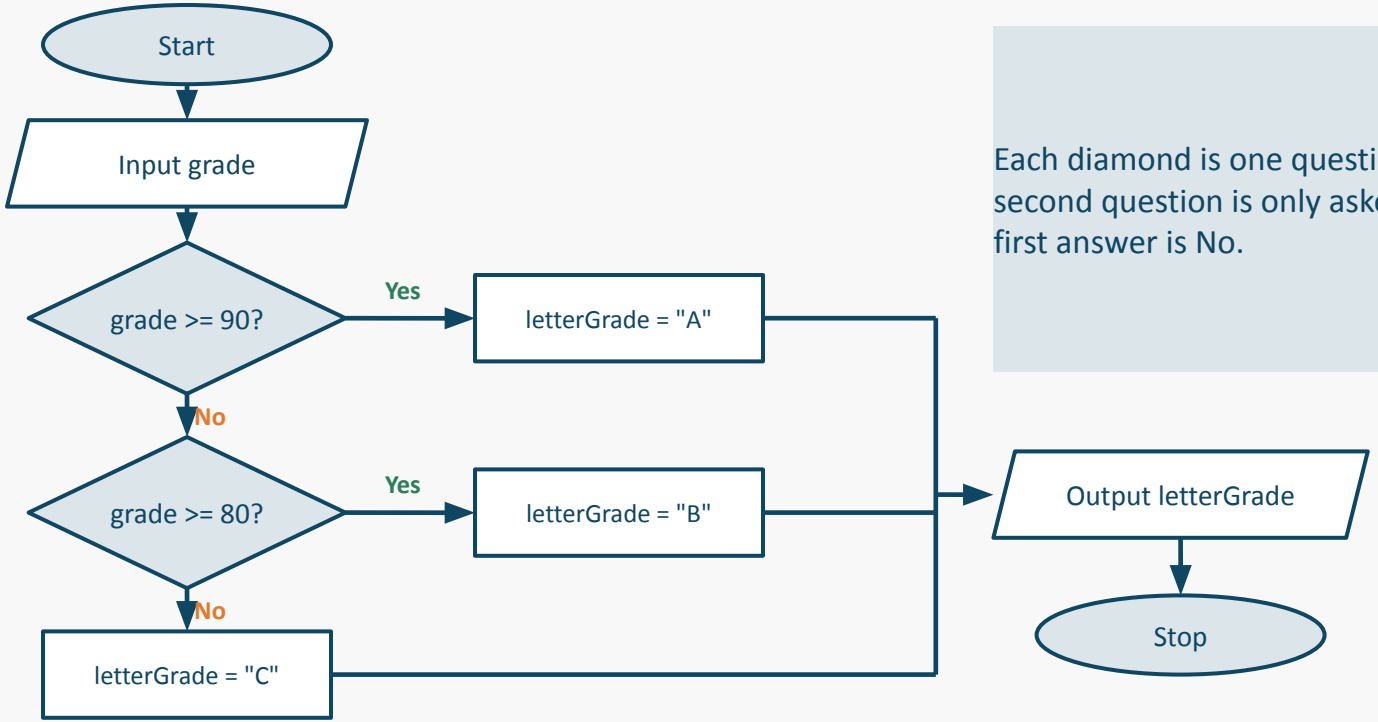
Why and?

Without it, a 95 would also pass grade >= 80 and get changed to "B". The and keeps B inside 80–89.

Notice

All three if checks run every time – even after we already know the grade is an A.

Nested IF-ELSE: Flowchart



Each diamond is one question. The second question is only asked when the first answer is No.

Program 2: Nested IF-ELSE

```
# Recitation problem 2
grade = int(input("Enter the integer numeric grade: "))

if grade >= 90:
    letterGrade = "A"
else:
    if grade >= 80:
        letterGrade = "B"
    else:
        letterGrade = "C"

print("Your letter grade is ", letterGrade)
```

No and needed

We only reach grade >= 80 when the grade is already below 90, so the upper limit is built in.

Same output

Program 1 and Program 2 give the same letter grade for every input – just a different structure.

Which Program Does Less Work?

Count the comparisons Python makes for each input

Input grade	Program 1 (separate ifs)	Program 2 (nested)
95 → A	4	**1**
85 → B	4	**2**
70 → C	4	**2**

Program 1 checks >= 90, >= 80, < 90 and < 80 every time.

1–2 vs 4

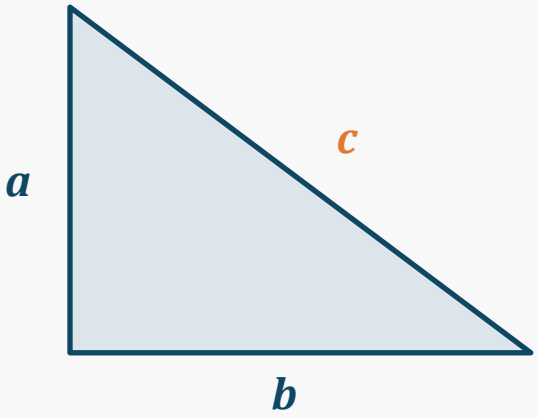
comparisons per input. Nested if-else stops as soon as it finds the answer → more efficient.

Pythagorean Theorem & import math

```
import math

aStr = input("Enter value for a: ")
a = float(aStr)
bStr = input("Enter value for b: ")
b = float(bStr)

c = math.sqrt(a**2 + b**2)
print("c = ", c)
```



$c = \sqrt{(a^2 + b^2)}$

- **`import math`** goes at the very top
- Without it → NameError: math is not defined
- **`**`** is the power operator (a**2 = a^2)

math.ceil() and math.floor()

math.ceil(x)

Smallest integer ≥ x ⟶ always rounds UP

math.floor(x)

Largest integer ≤ x ⟶ always rounds DOWN

Real use: 23 students, 5 per table ⟶ math.ceil(23 / 5) = 5 tables. You can't book 4.6 tables!

Try it in interactive mode

```
>>> import math
>>> math.ceil(4.2)
5
>>> math.floor(4.8)
4
>>> math.ceil(7.0)
7
>>> math.ceil(-4.2)
-4
>>> math.floor(-4.2)
-5
```

Ending a Program Early: quit()

```
aStr = input("Enter a positive value for a: ")
a = float(aStr)

if a <= 0:
    print("Input error")
    quit()

# only reached when a > 0
print(a)
```

- **`quit()`** stops the program right away
- Nothing after it runs
- Use it for bad input: negative length, zero people, etc.
- Always print a message first so the user knows why it stopped

Thank You

Questions?